

Conectividade C# (C Sharp) & PostgreSQL



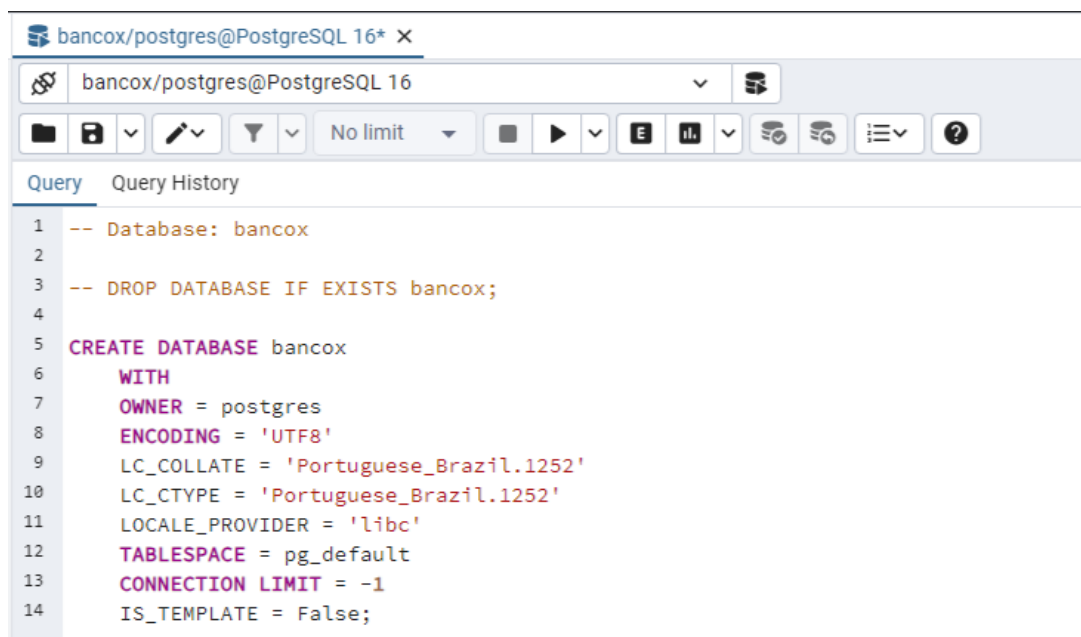
Atividade

Linguagem de Programação & Banco de Dados

Prof. Antonio Marcos Alvarez

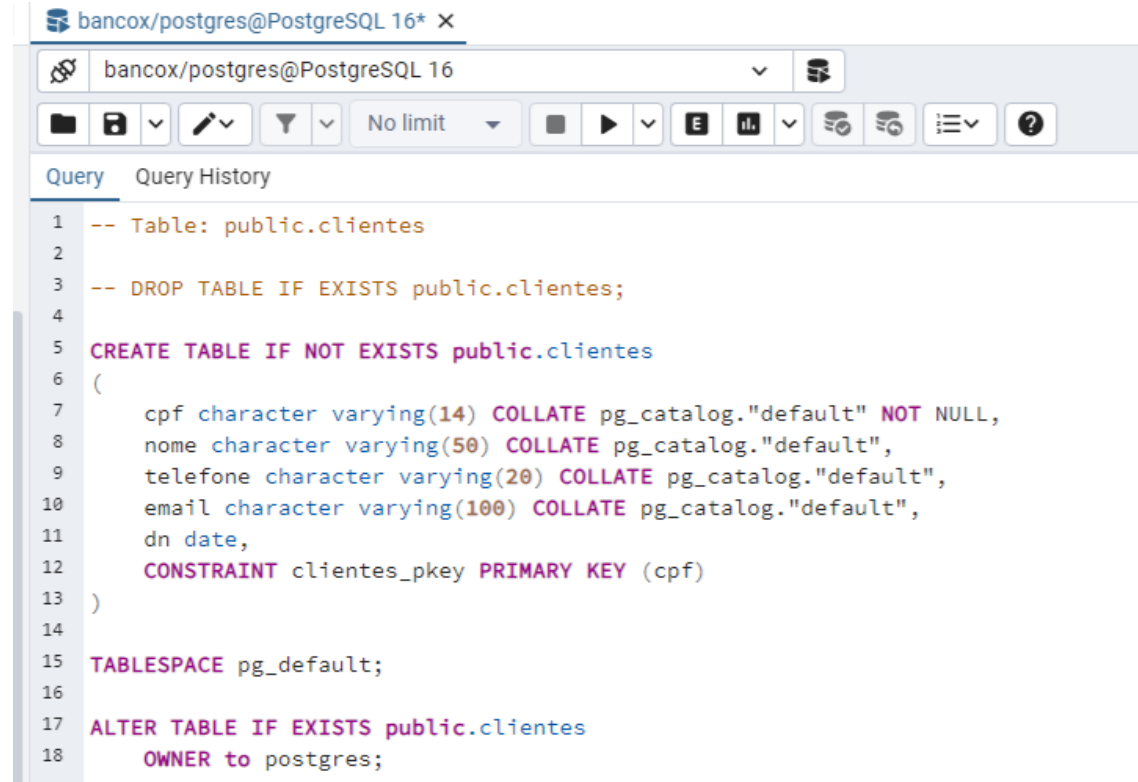
PostgreSQL

Analise através do **pgAdmin** (*Open Source – Interface Gráfica - Administrador e Plataforma de Desenvolvimento para o PostgreSQL*), a existência do banco de dados “bancox” e sua tabela “clientes”.



The screenshot shows the pgAdmin interface with the title bar 'bancox/postgres@PostgreSQL 16*'. The 'Query' tab is active, displaying a SQL script to create a database named 'bancox'. The script includes comments and various configuration options like encoding, collation, and tablespace.

```
1  -- Database: bancox
2
3  -- DROP DATABASE IF EXISTS bancox;
4
5  CREATE DATABASE bancox
6      WITH
7      OWNER = postgres
8      ENCODING = 'UTF8'
9      LC_COLLATE = 'Portuguese_Brazil.1252'
10     LC_CTYPE = 'Portuguese_Brazil.1252'
11     LOCALE_PROVIDER = 'libc'
12     TABLESPACE = pg_default
13     CONNECTION LIMIT = -1
14     IS_TEMPLATE = False;
```



The screenshot shows the pgAdmin interface with the title bar 'bancox/postgres@PostgreSQL 16*'. The 'Query' tab is active, displaying a SQL script to create a table named 'clientes' in the 'public' schema. The script includes comments and defines columns like cpf, nome, telefone, email, and dn, along with a primary key constraint.

```
1  -- Table: public.clientes
2
3  -- DROP TABLE IF EXISTS public.clientes;
4
5  CREATE TABLE IF NOT EXISTS public.clientes
6  (
7      cpf character varying(14) COLLATE pg_catalog."default" NOT NULL,
8      nome character varying(50) COLLATE pg_catalog."default",
9      telefone character varying(20) COLLATE pg_catalog."default",
10     email character varying(100) COLLATE pg_catalog."default",
11     dn date,
12     CONSTRAINT clientes_pkey PRIMARY KEY (cpf)
13 )
14
15 TABLESPACE pg_default;
16
17 ALTER TABLE IF EXISTS public.clientes
18     OWNER to postgres;
```

C# - Form1 (Cadastro de Clientes)

The diagram illustrates a Windows Form titled "Cadastro de Clientes" with the following components and their properties:

- Form1**
Text: Cadastro de Clientes
- mkt_CPF**
Mask: 999.999.999-99
- Txt_Nome**
MaxLength: 50
- Txt_Telefone**
MaxLength: 20
- Txt_Email**
MaxLength: 100
- mkt_DN**
Mask: 00/00/0000 (date)
- btn_Inserir**
Text: Inserir
- btn_Consultar**
Text: Consultar
- btn_Alterar**
Text: Alterar
- btn_Excluir**
Text: Excluir
- btn_Cancelar**
Text: Cancelar

The form layout includes input fields for CPF, Nome, Telefone, Email, and Data de Nascimento, along with buttons for Inserir, Consultar, Alterar, Excluir, and Cancelar.

Codificação (Impressão – PDF)

Observe o código, na íntegra:

//Npgsql é um ADO.NET Data Provider de código aberto para PostgreSQL

//ele permite que programas escritos em C#, Visual Basic, F#

//acessem o servidor de banco de dados PostgreSQL.

//Ele é implementado em código 100% C#, é gratuito e de código aberto.

```
using Npgsql;
```

```
using NpgsqlTypes; // Opcional
```

Link:

<https://www.npgsql.org/>

```
1 using System;
2 using System.Collections.Generic;
3 using System.ComponentModel;
4 using System.Data;
5 using System.Drawing;
6 using System.Linq;
7 using System.Text;
8 using System.Threading.Tasks;
9 using System.Windows.Forms;
10
11 //Npgsql é um ADO.NET Data Provider de código aberto para PostgreSQL
12 //ele permite que programas escritos em C#, Visual Basic, F#
13 //acessem o servidor de banco de dados PostgreSQL.
14 //Ele é implementado em código 100% C#, é gratuito e de código aberto.
15 using Npgsql;
16 using NpgsqlTypes;
17
18
19 namespace ex_ConectPostgreSQL_2025
20 {
21     public partial class Form1 : Form
22     {
23         public Form1()
24         {
25             InitializeComponent();
26         }
27
28         //Declarar Classe de Conexão e suas variaveis publicas
29         public NpgsqlConnection Conectar;
30         public NpgsqlCommand ComandoSQL;
31         public NpgsqlDataReader LerRegistro;
32         public string strCon =
33             "Server=localhost;Port=5432;Username=postgres;password=root;D
34             atabase=bancox";
35         public string strSQL = "";
36
37         //Não esquecer de Programar o Evento "Load" do Form1
38         private void Form1_Load(object sender, EventArgs e)
39         {
40             //Desabilitando Botões Alterar e Excluir (Ao abrir o Form)
41             btn_Alterar.Enabled = false;
42             btn_Excluir.Enabled = false;
43             btn_Cancelar.Visible = false;
44             mkt_CPF.Focus();
45         }
46
47         //Programação dos Eventos Click de cada Botão presente no Form1
48         private void btn_Inserir_Click(object sender, EventArgs e)
49         {
50             //Inserir Registro
51
52             //Bloco try/catch serve para tratamento de exceções
53             (possiveis falhas ou erros),
```

...	PostgreSQL_2025\ex_ConectPostgreSQL_2025\Form1.cs	2
-----	---	---

```

51      //Tratamento de códigos que podem não ser totalmente
      atendidos e gerarem alguma exceção/erro.
52      try
53      {
54          //Lembramos que estamos usando o Objeto para CPF
      (MaskedTextBox),
55          //quando usamos a propriedade Mask, invertemos o simbolo
      (,) por (.) e vice e versa
56          //Analisando a consistência (validação) do Objeto
      "mkt_CPF",
57          //onde não pode ser menor que 14 caracteres (extraindo
      os espaços vazios)
58          if (mkt_CPF.Text.Trim().Length == 14)
59          {
60              //Criar e Estanciar Objeto a Classe de Conexão com
      o Danco de Dados
61              Conectar = new NpgsqlConnection(strCon);
62
63              //Abrir Objeto de Conexão com Banco de Dados criada
      acima;
64              Conectar.Open();
65
66              //Montando a variavel tipo String "strSQL" de
      Leitura de Dados (concatenando) com os objetos do
      Formulário
67              strSQL = "SELECT * FROM clientes WHERE CPF = '" +
      mkt_CPF.Text + "'";
68
69              //MessageBox.Show(message, title, buttons)
70              //Mensagem para apresentar a String (strSQL)
71              MessageBox.Show(strSQL);
72
73              //Instanciando o Objeto de classe de Command
      (comando) para armazenar a Instrução / Clausulas SQL
74              ComandoSQL = new NpgsqlCommand(strSQL, Conectar);
75
76              //Executando comando com resposta de Leitura de
      Registros (linha por Linha)
77              LerRegistro = ComandoSQL.ExecuteReader();
78
79              //Metodo Read(): Informe um "boolean"
      "True" (Existe Registro) e "False" (NÃO Existe
      Registro),
80              //Possibilita Ler o Proximo Registro de uma Tabela
      (Enquanto for True, Se existir registro)
81
82              if (LerRegistro.Read())
83              {
84                  MessageBox.Show("Cliente já; Existe!!!",
      "Sistema Informa");
85              }
86              else
87              {

```

```

88          //Fechar Classe DataReader e Dispose (limpar o
Objeto) da Classe de ComandoSQL
89          LerRegistro.Close();
90          ComandoSQL.Dispose();
91          ComandoSQL.Transaction = null;
92
93          //Podemos efetuar a validação de todos os
campos, antes de inserirmos o registro...
94          //Montando a variável tipo String "strSQL" de
Inserção dos Dados (concatenando) com os objetos do
Formulário
95          strSQL = "INSERT INTO clientes " +
96                  "(cpf, nome, telefone, email, dn) " +
97                  "VALUES (" +
98                  "'" + mkt_CPF.Text + "'," +
99                  "'" + txt_Nome.Text + "'," +
100                 "'" + txt_Telefone.Text + "'," +
101                 "'" + txt_Email.Text + "'," +
102                 "TO_DATE ('" + mkt_DN.Text + "', 'DD/
MM/YYYY'))";
103
104          //Mensagem para apresentar a String (strSQL)
105          MessageBox.Show(strSQL);
106
107          //Criar o Objeto com a classe de Command
(comando) para armazenar a Instrução / Comando SQL
108          //comandoSQL já está definida como uma Classe
NpgsqlCommand, portanto só precisamos Instaciá-la;
109          ComandoSQL = new NpgsqlCommand(strSQL,
Conectar);
110
111          //Executando sem resposta
112          ComandoSQL.ExecuteNonQuery();
113          MessageBox.Show("Registro Inserido com
Sucesso...", "Sistema Informa");
114
115
116          //Limpar Objetos usando a função
117          LimparObjetos();
118      }
119
120          //Fechar Conexão
121          Conectar.Close();
122
123      }
124      else
125      {
126          MessageBox.Show("Preencher corretamente pelo menos
o campo CNPJ!!!");
127      }
128
129      //Votar Cursor para o Objeto de Formulário
130      mkt_CPF.Focus();

```

```
131
132
133     }
134     catch (Exception erro)
135     {
136         MessageBox.Show(erro.Message);
137     }
138
139 }
140
141 private void btn_Consultar_Click(object sender, EventArgs e)
142 {
143     //Bloco try/catch serve para tratamento de exceções           ↗
144     //Tratamento de códigos que podem não ser totalmente         ↗
145     //atendidos e gerarem alguma excessão/erro.
146     try
147     {
148         if (mkt_CPF.Text.Trim().Length == 14)
149         {
150             //Instanciar Objeto da Classe de Conexão com o         ↗
151             //Banco de Dados
152             Conectar = new NpgsqlConnection(strCon);
153
154             //Abrir Objeto de Conexão com Banco de Dados           ↗
155             criada acima;
156             Conectar.Open();
157
158             //Mondando a String (concatenando) com os objetos     ↗
159             //do Formulário
160             strSQL = "SELECT * FROM Clientes WHERE (CPF =" +      ↗
161             "'" + mkt_CPF.Text + "'"");
162
163             //Mensagem para apresentar a String (strSQL)
164             MessageBox.Show(strSQL);
165
166             //Instanciando o Objeto da classe de Command           ↗
167             //comando) para armazenar a Instrução / Clausulas     ↗
168             SQL
169             ComandoSQL = new NpgsqlCommand(strSQL, Conectar);
170
171             //OleDbDataReader rs = new OleDbDataReader             ↗
172             (comandoSQL);
173             //Executando sem resposta
174             //comandoSQL.ExecuteNonQuery();
175
176             LerRegistro = ComandoSQL.ExecuteReader();
177
178             //Metodo Read(): Informe um "boolean" "True" (exite ↗
179             Registro) e "False" (Não Existe Registro),
180             //Possibilita Ler o Proximo Registro de uma Tabela ↗
181             (Enquanto for True, Se existir registro)
182
183         }
184     }
185 }
```



```
173         if (LerRegistro.Read())
174         {
175             //Populando Objetos do Form com Dados do Registro (lerRegistro)
176             mkt_CPF.Text = LerRegistro.GetString(0);
177             txt_Nome.Text = LerRegistro.GetString(1);
178             txt_Telefone.Text = LerRegistro.GetString(2);
179             txt_Email.Text = LerRegistro.GetString(3);
180             mkt_DN.Text = LerRegistro.GetDateTime
181             (4).ToString("dd/MM/yyyy");
182
183             // Opção de Edição
184
185             if (DialogResult.No == MessageBox.Show("Deseja
186             Editar Registro?", "Sistema Informa",
187             MessageBoxButtons.YesNo,
188             MessageBoxIcon.Question,
189             MessageBoxDefaultButton.Button2))
190             {
191                 //Limpar Objetos do Formulário usando a
192                 função
193                 LimparObjetos();
194                 //Desativa Botões
195                 btn_Alterar.Enabled = false;
196                 btn_Excluir.Enabled = false;
197                 //Ativa CPF (Edição)
198                 mkt_CPF.Enabled = true;
199             }
200             else
201             {
202                 //Ativa Botões
203                 btn_Alterar.Enabled = true;
204                 btn_Excluir.Enabled = true;
205                 //Desativa Botões
206                 btn_Inserir.Enabled = false;
207                 btn_Consultar.Enabled = false;
208                 //Desativa CPF (Edição)
209                 mkt_CPF.Enabled = false;
210                 //Visualiza
211                 btn_Cancelar.Visible = true;
212             }
213         }
214         else
215         {
216             MessageBox.Show("CPF " + mkt_CPF.Text + " Não
217             Localizado!!!", "Sistema Informa");
218             mkt_CPF.Focus();
219         }
220     }
```

```
219
220         //Fechar Classe DataReader e Dispose (limpar o Objeto) da Classe de ComandoSQL
221         LerRegistro.Close();
222         ComandoSQL.Dispose();
223         ComandoSQL.Transaction = null;
224
225         //Fechar Conexão
226         Conectar.Close();
227
228     }
229     else
230     {
231         MessageBox.Show("Preencher corretamente pelo menos o campo CPF!!!");
232     }
233
234     //Votar Cursor para o Objeto de Formulario
235     mkt_CPF.Focus();
236
237 }
238
239 catch (Exception erro)
240 {
241     MessageBox.Show(erro.Message);
242 }
243
244
245 private void btn_Alterar_Click(object sender, EventArgs e)
246 {
247     try
248     {
249         if (DialogResult.Yes == MessageBox.Show("Confirma Alteração do Registro?", "Sistema Informa",
250             MessageBoxButtons.YesNo,
251             MessageBoxIcon.Question,
252             MessageBoxDefaultButton.Button2))
253         {
254             //Instanciar Objeto da Classe de Conexão com o Banco de Dados
255             Conectar = new NpgsqlConnection(strCon);
256
257             //Abrir Objeto de Conexão com Banco de Dados criada acima;
258             Conectar.Open();
259
260             //Mondando a String (concatenando) com os objetos do Formulário
261             strSQL = "UPDATE Clientes SET " +
                "nome = '" +
                txt_Nome.Text + "', " +
                "telefone = '" +
                txt_Telefone.Text + "', " +
```

```

...PostgreSQL_2025\ex_ConectPostgreSQL_2025\Form1.cs 7
262 "email = '" +
txt_Email.Text + "', " +
263 "dn = TO_DATE ('" +
mkt_DN.Text + "', 'DD/MM/YYYY') " +
264 " WHERE cpf = '" +
mkt_CPF.Text + "' ";
265
266 //Mensagem para apresentar a String (strSQL)
267 MessageBox.Show(strSQL);
268
269 //Instanciando o Objeto da classe de Command
(comando) para armazenar a Instrução / Clausulas SQL
ComandoSQL = new NpgsqlCommand(strSQL, Conectar);
270
271 //Executando sem resposta
272 ComandoSQL.ExecuteNonQuery();
273
274 MessageBox.Show("Registro Alterado com Sucesso...",
"Sistema Informa");
275
276 //Limpar Objetos do Formulário usando a função
277 LimparObjetos();
278
279 //Destativa Botões
280 btn_Alterar.Enabled = false;
281 btn_Excluir.Enabled = false;
282
283 //Ativa Botões
284 btn_Consultar.Enabled = true;
285 btn_Inserir.Enabled = true;
286
287 //Esconde / Não Visusaliza
288 btn_Cancelar.Visible = false;
289
290 //Ativa CPF (Edição)
291 mkt_CPF.Enabled = true;
292
293 //Volta cursor ao objeto CPF
294 mkt_CPF.Focus();
295
296 //Fechar Conexão
297 Conectar.Close();
298 }
299 else
300 {
301 //Volta para edição
302 txt_Nome.Focus();
303 }
304 }
305 catch (Exception erro)
306 {
307 MessageBox.Show(erro.Message);
308 }
309 }

```

```
310
311     private void btn_Excluir_Click(object sender, EventArgs e)
312     {
313         try
314         {
315             if (DialogResult.Yes == MessageBox.Show("Confirma
316                 Exclusão do Registro?", "Sistema Informa",
317                     MessageBoxButtons.YesNo,
318                     MessageBoxIcon.Question,
319                     MessageBoxDefaultButton.Button2))
320             {
321                 //Instanciar Objeto da Classe de Conexão com o
322                 Danco de Dados
323                 Conectar = new NpgsqlConnection(strCon);
324
325                 //Abrir Objeto de Conexão com Banco de Dados
326                 criada acima;
327                 Conectar.Open();
328
329                 //Mondando a String (concatenando) com os objetos
330                 do Formulário
331                 strSQL = "DELETE FROM Clientes WHERE cpf = '" +
332                     mkt_CPF.Text + "' ";
333
334                 //Mensagem para apresentar a String (strSQL)
335                 MessageBox.Show(strSQL);
336
337                 //Instanciando o Objeto da classe de Command
338                 (comando) para armazenar a Instrução / Clausulas SQL
339                 ComandoSQL = new NpgsqlCommand(strSQL, Conectar);
340
341                 //Executando sem resposta
342                 ComandoSQL.ExecuteNonQuery();
343                 MessageBox.Show("Registro Excluído com Sucesso...",
344                     "Sistema Informa");
345             }
346             else
347             {
348                 MessageBox.Show("Operação Cancelada...", "Sistema
349                     Informa");
350             }
351
352             //Limpar Objetos do Formulário usando a função
353             LimparObjetos();
354
355             //Destaiava Botões
356             btn_Alterar.Enabled = false;
357             btn_Excluir.Enabled = false;
358
359             //Ativa Botões
360             btn_Consultar.Enabled = true;
361             btn_Inserir.Enabled = true;
```

```
353         //Esconde / Não Visualiza
354         btn_Cancelar.Visible = false;
355
356         //Ativa CPF (Edição)
357         mkt_CPF.Enabled = true;
358
359         //Volta cursor ao objeto CPF
360         mkt_CPF.Focus();
361
362         //Fechar Conexão
363         Conectar.Close();
364     }
365     catch (Exception erro)
366     {
367         MessageBox.Show(erro.Message);
368     }
369 }
370
371 private void btn_Cancelar_Click(object sender, EventArgs e)
372 {
373     //Limpar Objetos do Formulário usando a função
374     LimparObjetos();
375
376     //Desativa Botões
377     btn_Alterar.Enabled = false;
378     btn_Excluir.Enabled = false;
379
380     //Ativa Botões
381     btn_Inserir.Enabled = true;
382     btn_Consultar.Enabled = true;
383
384     //Ativa CPF (Edição)
385     mkt_CPF.Enabled = true;
386
387     //Volta cursor ao objeto CPF
388     mkt_CPF.Focus();
389
390     //Fechar Conexão
391     Conectar.Close();
392     btn_Cancelar.Visible = false;
393 }
394
395 //Criando Função LimparObjetos
396 public void LimparObjetos()
397 {
398     //Limpar Objetos do Formulário
399     mkt_CPF.Clear();
400     txt_Nome.Clear();
401     txt_Telefone.Clear();
402     txt_Email.Clear();
403     mkt_DN.Clear();
404
405     return;
```

```
406         }  
407  
408     }  
409 }  
410
```